Odds and Evens Game - Report

Martin Tomassi

1 Problem analysis

The assignment requires designing a tournament-style Odds and Evens game for $N = 2^m$ players, structured into m sequential elimination rounds. From a concurrent programming perspective, the problem is defined by three main requirements. First, **intra-round parallelism** dictates that all $N/2$ matches within a given round are mutually independent. Because no match relies on the progress or outcome of any other match in the same round, these matches can be played concurrently. Second, **inter-round synchronization** imposes a strict barrier between consecutive rounds: a round cannot start until every match in the preceding round has concluded and produced a winner. Finally, **the total absence of shared state** requires that players, match referees, and the overall tournament progression coordinate exclusively through the synchronous exchange of messages.

2 General strategy

The implementation is structured around long-lived player goroutines, short-lived match-specific referee goroutines and a round coordinator that manages the tournament round by round. All coordination between these entities relies exclusively on synchronous message passing through unbuffered channels, eliminating the need for shared state. Players are instantiated only once at the start of the tournament and remain active for its entire duration. Each player goroutine runs a reactive loop, remaining idle until it receives a play request; at that point, it selects a random number and returns it via its dedicated channel. Winning players advance from round to round simply by passing their channel structures on to subsequent matches, keeping their goroutines active without any external management, while eliminated players wait safely in their request channel loop until the tournament ends. Each match within a round is handled by a dedicated referee goroutine spawned by the round coordinator. A referee interacts with two players using synchronous message passing: it signals both players to take their turn, waits for the numbers chosen by both, calculates the winner according to the **Odds and Evens** rule and reports the winning player to the coordinator. The round coordinator manages intra-round execution by spawning a referee goroutine for each independent pair of players. It uses a `sync.WaitGroup` along with a shared results channel to track active referees. A goroutine waits for all referee tasks to complete before closing the results channel. Because the coordinator iterates over this channel until it is drained and closed, the channel reading itself acts as a synchronisation barrier, guaranteeing that all matches in the current round conclude before the next round begins. Once the overall winner is declared, the main process initiates a graceful termination protocol by closing the request channels of all original players. This signals every long-lived player goroutine to exit its loop and terminate cleanly.